

Project 04 — On-Device Face Verification on ExecuTorch

Mobile · consented 1:1 verification · biometric templates never leave the device

Contents

Project 04 — On-Device Face Verification (ExecuTorch)	1
0. Numbered build plan	1
1. Scope, stated first	2
2. Why ExecuTorch for this workload	2
3. There is no KV cache here — and that is the point	3
4. Models and compression	3
5. Architecture	3
6. Frontend	4
7. Architecture graph: node inventory	5
8. The improvement loop for this project	6
8a. LoRA adapters: where they earn their place here	6
9. The release gate for this project	7
10. Monitoring and KPIs	7
11. Security, audit and compliance	8
12. Deployment	8
13. Deliverables	9
14. Out of scope	9

Project 04 — On-Device Face Verification (ExecuTorch)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code’s plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **ExecuTorch serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.

6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer’s own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

1. Scope, stated first

This project builds **1:1 face verification** — confirming that the person in front of the camera is the enrolled owner of *this* device or *this* credential. It is the same category of function as a fingerprint unlock or a photo-matched building pass.

It is **not** a 1:N identification system, not a surveillance tool, and not a watchlist matcher. Three constraints enforce that in the design, and they are architectural rather than policy:

1. **Enrolment is explicit and consented**, initiated by the subject on the device.
2. **Templates never leave the device.** There is no central gallery to search against, so 1:N identification of strangers is not merely disallowed — it is not possible with what gets built.
3. **The enrolled set is one person per credential.** The matching API takes a probe and a single stored template. There is no “search all enrolled” call.

Build it this way. If a requirement arrives later to identify unknown people against a central database, that is a materially different system with different legal obligations, and it should be scoped, reviewed and consented separately.

Target user: workforce access control, secure app unlock, device-bound identity for regulated workflows — cases where the alternative is a shared PIN or a badge that can be handed over.

2. Why ExecuTorch for this workload

- **It is PyTorch’s on-device runtime**, so the path from a trained model to a deployed one is short: export to .pte from the same PyTorch graph, without a lossy conversion through an intermediate format. For a vision pipeline that may need retraining on customer data, that shortens the loop considerably.
 - **Delegate-based backends.** XNNPACK for CPU, Core ML and Metal on Apple, Vulkan and Qualcomm QNN on Android — chosen per device at export time, with the same source graph. A face embedding runs on the NPU where one exists and falls back gracefully where it does not.
 - **Small runtime footprint.** ExecuTorch is designed for a constrained binary, which matters when the model must ship inside an app.
 - **Ahead-of-time compilation and memory planning.** Buffer sizes are resolved at export, so runtime allocation is predictable — which is what makes per-frame latency stable rather than merely fast on average.
-

3. There is no KV cache here — and that is the point

Per shared requirements §4, state this explicitly on the architecture node rather than quietly omitting it:

A face embedding model runs **one forward pass per image**. There is no autoregressive generation, therefore no KV state, therefore no KV cache and no continuous batching. Fabricating one would be dishonest, and a client with domain knowledge will notice.

The optimisations that *do* apply, and which must be demonstrated in their place:

Technique	What it buys
INT8 post-training quantization	~4× smaller, faster on NPU/DSP which are integer-native
Quantization-aware training	Recovers accuracy INT8 PTQ loses — worth it, because face embeddings are sensitive to precision
Operator fusion	Conv+BN+ReLU folded — fewer memory round trips, which is the actual bottleneck
Delegate selection	NPU vs GPU vs CPU per device tier
Input resolution tuning	The single largest latency lever in vision, and a direct accuracy trade

That table is the honest analogue of the vLLM project's KV-cache story, and it belongs in the architecture graph in the same position.

4. Models and compression

Stage	Model	Notes
Face detection	BlazeFace or RetinaFace-mobile	Fast, single-face is enough
Landmark + alignment	5-point	Alignment before embedding matters more than model size
Embedding	MobileFaceNet or ArcFace (MobileNetV3 backbone)	128- or 512-d, L2-normalised
Liveness	Small anti-spoofing CNN	Mandatory — see §7

Compression spec - Export each stage to .pte with INT8 quantization. Calibrate on a **diverse** face dataset — see the fairness note below, because this is where bias enters. - Record per stage and per device tier: model size, latency, accuracy delta. - Choose delegates per target and record the fallback chain.

Quality gate — and this one carries an obligation the others do not - Verification accuracy: TAR at a fixed FAR (e.g. TAR@FAR=1e-4), gated as a delta against the FP32 baseline. - **Fairness evaluation across demographic groups, as a gating criterion, not a report.** Face recognition accuracy is well documented to vary by skin tone, age and gender. A model that passes on aggregate while failing badly for one group is not shippable, and aggregate-only evaluation is how that ships anyway. Gate on the *worst-group* metric, not the mean. - Liveness: spoof detection rate against printed photos and screen replays. - On-device latency budget per device tier.

Run the gate on target hardware. INT8 kernels differ across delegates.

5. Architecture

User (Product tab)

- Camera capture
- Device authentication (passcode fallback)
- Authorization (which credential is being verified)
- Face detection → landmark alignment

- LIVENESS CHECK ← before embedding, always
- ExecuTorch embedding model
 - INT8 quantized .pte
 - NPU / GPU / CPU delegate
 - operator fusion
- 1:1 match against the single stored template (cosine, on-device)
- Confidence gate → accept / reject / fall back to passcode
- Local structured logs (no images, no templates)
- Local append-only audit log, hash-chained
- Metrics buffer ... opportunistic sync ...
- [Backend] Prometheus → Grafana
- Model registry → staged OTA rollout rings ← replaces Kubernetes
- Release rings (signed artifacts, canary devices)

== IMPROVEMENT LOOP (the RLHF analogue) =====

- False-reject and false-match reports from operator review
- embeddings and similarity SCORES only – never the frames
 - Threshold review: projected TAR@FAR from stored score distributions. This is the loop that improves the product day to day, and it needs no training run at all
 - Re-enrolment queue for poor-quality enrolments
 - Promotion gate: TAR@FAR suite (per demographic group) + on-device latency / energy harness

==> back to the embedding model

There is no preference fine-tune here by default, and that is deliberate: consent to be recognised is not consent to become training data.

Build deliberately:

- **The template is stored in the platform secure enclave** (Secure Enclave / StrongBox), never in app storage, never in a backup, never transmitted. This is the architectural claim the whole project rests on, and it must be true.
- **Liveness runs before embedding, unconditionally.** A verification system without anti-spoofing is defeated by a photograph, which makes it worse than a PIN because it carries an unearned appearance of security.
- **Passcode fallback always exists.** Biometrics fail — injuries, lighting, a changed appearance. A system with no fallback locks legitimate users out, and the failure correlates with exactly the groups the model already serves worst.
- **Images are never persisted.** Capture, embed, compare, discard. The audit log records that a verification occurred and its outcome, never the face.

6. Frontend

Product tab — enrolment flow with explicit consent, then verification. Show live: detection box, liveness verdict, match score against threshold, and the decision. Include a deliberate **spoofing demo** — hold a photo to the camera and watch liveness reject it. That is the most persuasive twenty seconds in the video.

Configuration: match threshold with the FAR/FRR trade shown as a curve rather than a number, liveness strictness, fallback policy, enrolment management with deletion.

Architecture tab — 3D graph. Logos: PyTorch/ExecuTorch, Core ML, Qualcomm, Android, Prometheus, Grafana. Strongest copy on **INT8 quantization**, **delegate selection**, **liveness**, **secure enclave storage**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only

one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and 00-shared-requirements.md §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

7. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the ArchNode shape in 00-shared-requirements.md §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Subject	neutral	— (the person being recognised, who has rights here)
1	Capture UI	teal	Camera preview, enrolment flow, consent capture, result display
2	Consent & Liveness Gate	amber	Explicit consent record, presentation-attack detection, on-device sandbox, no network path for imagery
3	Detect & Align	violet	Face detection, landmark alignment, quality scoring, rejection of unusable frames
4	ExecuTorch Runtime	orange	MobileFaceNet / ArcFace embedding model (INT8 PTQ) , XNNPACK / Core ML delegation, operator fusion, calibration set, “no KV cache — and why”
5	Matching	violet	Enrolment index, cosine similarity search, threshold policy, ambiguity refusal
6	Local Store	blue	Encrypted template store — templates only, never images , keystore-backed
7	Monitoring & Audit	magenta	Biometric access log (hash-chained), consent records, erasure requests, opt-in aggregated metrics

#	Stage card	Colour role	Expands into
8	Improvement Loop	green	False-match / false-reject reporting, threshold tuning from operator review, re-enrolment queue, promotion gate □ accuracy suite, on-device latency harness
9	Release Rings	grey	Signed model artifacts, staged OTA rings, canary devices, rollback

Two cards carry this project, and neither is accuracy. **Consent & Liveness Gate** is what makes the system deployable at all in a jurisdiction that treats faces as special-category data, and **Local Store** — templates, never images — is the single most reassuring sentence you can put in front of a buyer's legal team.

The no KV cache child under the runtime is deliberate. Keep it as a node with its detail panel explaining that a recognition model runs one forward pass per frame and has no autoregressive state to cache, and naming INT8 PTQ and delegate selection as the real optimisation. A client who knows the domain will notice a fabricated KV cache immediately.

8. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

Biometrics invert the usual feedback economics: **the errors are the asset and the data is radioactive.**

- **False-reject reports** — the subject was enrolled and was not recognised. Capture the *embedding and the similarity score*, never the frame.
- **False-match reports** — someone was recognised as someone else. Rare, far more serious, and the one that must page a human.
- **Threshold review** — the operator adjusts the accept threshold and sees the projected effect on TAR@FAR from stored score distributions. This is the loop that actually improves the product day to day, and it needs no training run.

Hard rules, and they are not negotiable:

- **Never train on captured faces without separate, explicit, revocable consent for that specific purpose.** Consent to be recognised is not consent to become training data, and treating it as such is the failure mode that ends deployments.
- **Store score distributions, not imagery.** Threshold tuning needs the scores. It does not need the faces.
- **Erasure must remove the template, the scores and the enrolment.** Support GDPR Art. 17 as a working button, not a policy paragraph.

Most of the improvement here comes from threshold policy and enrolment quality, not from fine-tuning. Say that plainly rather than implying a training loop you would not actually be allowed to run.

8a. LoRA adapters: where they earn their place here

Be careful here — this is where LoRA is most often claimed and least often useful.

LoRA adapts a *training-time* graph. ExecuTorch consumes an exported, frozen, quantized graph. So the honest position is:

- **LoRA is a training technique in this project, not a serving one.** If you adapt the embedding backbone to a client's population, you do it with LoRA in PyTorch, then **merge the adapter into the base weights before export and quantization**. What ships to the device has no adapter in it.
- **There is no per-request adapter swapping on device.** Anything claiming otherwise for a mobile embedding model is describing a system that does not exist. Do not put an adapter-routing node in this architecture graph.

Where it is genuinely worth testing: domain adaptation of the embedding backbone when the deployment population differs markedly from the training distribution — a specific workforce, camera geometry or lighting environment. LoRA makes that adaptation cheap to try and cheap to abandon, which matters because most such attempts should be abandoned.

And a hard constraint that outranks the technique. Adapting a face model to a population means training on that population's faces, which requires separate, explicit, revocable consent for that specific purpose. Consent to be recognised is not consent to become training data. If that consent is not obtainable — and often it is not — the correct engineering answer is to improve enrolment quality and threshold policy instead, and to say so rather than quietly training anyway.

If you do adapt: re-run the full gate on the merged, re-quantized artifact, including the per-demographic breakdown. A backbone adapted to one population can lose accuracy on another, and an aggregate number will hide it.

9. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out verification suite reporting TAR @ FAR = 1e-4 and 1e-6 , plus per-demographic breakdown	A single accuracy number is inadequate and, here, irresponsible. Report accuracy per demographic group ; a compression step that costs 2 points overall may cost 8 in one group, and shipping that is both a product failure and a legal one
Performance	Is it still fast and cool enough?	On-device harness: p95 end-to-end latency, peak memory, energy per recognition, sustained-throughput thermal behaviour	Recognition happens in a queue of real people. p95 above roughly 300 ms is felt as hesitation

1m_eval and GuideLLM do not apply — there is no language model and no serving endpoint. Substitute the two harnesses above and state the substitution explicitly, as section 4 of the shared requirements demands.

10. Monitoring and KPIs

Section	Metrics
Quality	TAR@FAR, false accept and false reject rates, broken out by demographic group , liveness detection rate, fallback-to-passcode rate
Performance	End-to-end verification latency p50/p95/p99, per-stage breakdown, model size, peak RAM, battery per verification, delegate in use, thermal events
Security	Spoof attempts detected, failed verifications per credential (credential-stuffing analogue), enclave access failures, model integrity failures
Audit	Every enrolment, verification, deletion: outcome, model version, threshold — never the image or template
Improvement	False-reject and false-match reports, score distribution drift, projected TAR@FAR at the current threshold, re-enrolment queue depth — per demographic group

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The fairness breakdown belongs on the dashboard, not in a one-time report. Accuracy drift affects groups unevenly, and a system that was fair at launch can stop being fair as the deployed population changes. If it is not measured continuously it is not managed.

11. Security, audit and compliance

This project carries the heaviest compliance load in the pack. Face templates are **special-category personal data under GDPR Art. 9** and generally require explicit consent, with several jurisdictions imposing additional biometric-specific statutes.

- **Explicit, informed, revocable consent** captured at enrolment and logged. Not a checkbox in a EULA — a specific consent to biometric processing that states what is stored, where, and for how long.
- **Erasure (Art. 17) must be immediate and complete.** Deleting the template from the enclave is the entire deletion, because there is no other copy. That is a strong property — make it demonstrable in the demo.
- **Data minimisation:** store the template, never the image. A template is not reversible to a photograph in the way a stored image obviously is.
- **On-device processing means no international transfer** and no third-party processor for the biometric step. That is a genuinely simpler compliance posture and worth stating precisely.
- **Audit log** records verification events and outcomes, hash-chained, containing no biometric data itself.
- **Retention:** define it explicitly, including what happens when an employee leaves or a device is de-commissioned.

Do not claim certification. Show the controls and let the client's counsel assess them.

12. Deployment

Same shape as Project 03: signed .pte artifacts versioned independently of the app binary, staged OTA rollout rings with halt and rollback, TestFlight and Play testing tracks as the staging → production path.

Additional obligation here: a model update changes matching behaviour. A threshold that was correctly calibrated for the old embedding may be wrong for the new one, silently raising the false accept rate. Re-

run the quality gate including the fairness evaluation before any rollout, and treat threshold recalibration as part of the model release, not a follow-up.

13. Deliverables

- Enrolment and verification flows, fully on-device
- Liveness detection with a working spoof demo
- ExecuTorch export pipeline with INT8 quantization and delegate selection
- Quality gate including **worst-group fairness gating**
- All three tabs
- Local hash-chained audit log with consent records
- Signed model artifacts with staged rollout
- Measured accuracy and latency across at least two device tiers

Acceptance: 1. Verification completes in under 500 ms p95 on a mid-range device 2. Printed photo and screen replay are both rejected by liveness 3. Fairness gate fails when a deliberately skewed model is supplied 4. Deleting an enrolment removes the template from the enclave, verifiably 5. No image or template appears in any log, packet capture or backup 6. Architecture tab and guided tour run offline

14. Out of scope

1:N identification against a gallery — explicitly excluded, see §1. Surveillance, covert capture, or any operation without the subject's active participation. Emotion or demographic inference from faces. Age estimation. Cross-device template sync, which would require a central store and undermine the architecture.